

GitHub Actions

Interview Bible — From Fundamentals to Senior-Level Mastery

DevOps • SRE • Cloud Operations

A complete reference — concepts, workflow syntax, integrations, troubleshooting, common errors, tricky questions & expected Q&A for candidates with 5+ years of experience

Table of Contents

1. GitHub Actions Fundamentals
2. Workflow Syntax Deep Dive
3. Actions: Types, Marketplace & Reusability
4. Runners: Hosted, Self-Hosted & Scaling
5. Secrets, Environments & OIDC
6. Security & Permissions Hardening
7. Caching, Artifacts & Performance
8. Integrations (Docker, Kubernetes, Terraform, Cloud, ChatOps)
9. Monorepos, Concurrency & Cost Optimization
10. Common Errors & Troubleshooting Playbook
11. Tricky Questions & Traps
12. Expected Interview Q&A (Basic → Advanced)
13. System Design Style Questions
14. Behavioral / Experience-Based Questions
15. Quick Reference Cheat Sheet

1. GitHub Actions Fundamentals

1.1 What Is GitHub Actions

GitHub Actions is GitHub's native CI/CD and automation platform. Automations are defined as **workflows** (YAML files in `.github/workflows/`) that respond to repository **events** and run **jobs** made of **steps**, executed on **runners**. Because it's built into GitHub, it also automates non-CI tasks: issue triage, PR labeling, release notes, stale-issue closing, ChatOps.

1.2 Core Building Blocks

Concept	Description
Workflow	A YAML file describing automation; one repo can have many workflows
Event	What triggers a workflow (<code>push</code> , <code>pull_request</code> , <code>schedule</code> , <code>workflow_dispatch</code> , <code>release</code> , <code>issue_comment</code> , etc.)
Job	A set of steps that run on the same runner; jobs run in parallel unless <code>needs:</code> creates a dependency
Step	A single task: either a shell command (<code>run:</code>) or an action (<code>uses:</code>)
Action	A reusable unit of automation (JavaScript, Docker, or composite) — the "function call" of GHA
Runner	The VM/container that executes a job: GitHub-hosted or self-hosted

1.3 Minimal Workflow

```
name: CI
on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "Hello CI"
```

1.4 Jobs Run in Parallel by Default Senior favorite

A common trap: candidates assume jobs in one workflow file run sequentially top-to-bottom like steps do. In fact, jobs run in parallel unless explicitly ordered with `needs:` . Only steps within the same job are strictly sequential.

```
jobs:
  lint:
    runs-on: ubuntu-latest
    steps: [ ... ]
  test:
    runs-on: ubuntu-latest
    steps: [ ... ]
  build:
    needs: [lint, test]      # waits for both to succeed
    runs-on: ubuntu-latest
    steps: [ ... ]
```

1.5 Where Actions Fits in the Broader CI/CD Landscape

GitHub Actions is a full CI/CD tool but is often paired with GitOps controllers (ArgoCD/Flux) for the deployment half in Kubernetes-heavy shops — Actions builds/tests/scans and updates a manifest repo; the GitOps controller reconciles the cluster. It competes directly with GitLab CI, CircleCI, and Jenkins, and its main differentiators are tight GitHub integration, the Marketplace ecosystem, and native OIDC federation to cloud providers.

2. Workflow Syntax Deep Dive

2.1 Triggers (`on:`)

Event	Use case
<code>push</code>	Run CI on every commit to specified branches/paths
<code>pull_request</code>	Run checks on PRs; runs with the fork's code and limited token permissions for external PRs
<code>pull_request_target</code>	Runs with base repo's permissions/secrets even for fork PRs — dangerous if misused (see section 6)
<code>workflow_dispatch</code>	Manual trigger from the UI/API, can accept typed inputs — used for manual deploys/runbooks
<code>schedule</code>	Cron-based, e.g. nightly builds; note: minimum granularity ~5 minutes and can be delayed under load
<code>workflow_call</code>	Makes the workflow reusable/callable from other workflows
<code>repository_dispatch</code>	Trigger from an external system via the API (e.g., another CI, a webhook receiver)
<code>release</code>	Trigger on GitHub release creation/publish — common for publishing steps

2.2 Filtering

```
on:
  push:
    branches: [main, 'release/**']
    paths: ['services/api/**']
    tags: ['v*']
```

`paths:` filtering is essential in monorepos to avoid running every service's pipeline on every commit.

2.3 Contexts & Expressions

Contexts expose data via `${{ }}` expression syntax: `github` (event/repo metadata), `env` , `secrets` , `vars` , `matrix` , `steps` (outputs from previous steps), `needs` (outputs from dependency jobs), `job` , `runner` .

```
steps:
- id: version
  run: echo "value=1.2.3" >> "$GITHUB_OUTPUT"
- run: echo "Version is ${{ steps.version.outputs.value }}"
```

2.4 Conditionals

```
steps:
- name: Deploy only on main
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  run: ./deploy.sh

- name: Always run cleanup even if a previous step failed
  if: always()
  run: ./cleanup.sh

- name: Only run on failure
  if: failure()
  run: ./notify-failure.sh
```

Trap: by default, if any step fails, all subsequent steps in that job are skipped. Use `if: always()` or `if: failure()` deliberately for cleanup/notification steps that must run regardless of outcome.

2.5 Matrix Builds

```
strategy:
  fail-fast: false
  matrix:
    os: [ubuntu-latest, windows-latest]
    node: [18, 20, 22]
  exclude:
```

```
- os: windows-latest
  node: 18

jobs:
  test:
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/setup-node@v4
        with: { node-version: '${{ matrix.node }}' }
```

`fail-fast: true` (default) cancels all other matrix jobs the moment one fails — set to `false` when you want full visibility into every combination's result (e.g., cross-platform compatibility testing).

2.6 Environment Variables vs Secrets vs Variables

Type	Visible in logs?	Use case
<code>env:</code>	Yes	Non-sensitive configuration
<code>secrets.*</code>	No (auto-masked)	Credentials, tokens, keys
<code>vars.*</code>	Yes	Non-sensitive configuration shared at org/repo/environment level

3. Actions: Types, Marketplace & Reusability

3.1 Three Types of Actions

Type	How it works	Pros/Cons
JavaScript action	Runs directly on the runner via Node.js	Fast (no container startup), but tied to Node runtime
Docker container action	Packages the action as a Docker image	Full environment control/any language, but slower (image pull/build) and Linux-runner only
Composite action	Bundles multiple existing steps/actions into one reusable action (YAML only, no code)	Simplest to author; great for standardizing a sequence of shell/action steps

3.2 Composite Action Example

```
# .github/actions/setup-and-test/action.yml
name: 'Setup and Test'
runs:
  using: 'composite'
  steps:
    - uses: actions/setup-node@v4
      with: { node-version: '20' }
    - run: npm ci
      shell: bash
    - run: npm test
      shell: bash
```

3.3 Reusable Workflows vs Composite Actions Senior favorite

Reusable Workflow (workflow_call)	Composite Action
Reuses entire jobs (can have multiple jobs, matrix, its own runs-on)	Reuses a sequence of steps within a single existing job
Called with uses: org/repo/.github/workflows/file.yml@v1	Called with uses: org/repo/.github/actions/name@v1
Best for standardizing an entire pipeline shape across many repos	Best for standardizing a repeated block of steps inside a job

```
# Reusable workflow definition: .github/workflows/deploy.yml
on:
  workflow_call:
    inputs:
      environment: { required: true, type: string }
    secrets:
      DEPLOY_TOKEN: { required: true }
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}
    steps:
      - run: ./deploy.sh
        env: { TOKEN: ${{ secrets.DEPLOY_TOKEN }} }

# Caller workflow
jobs:
  call-deploy:
    uses: my-org/shared-workflows/.github/workflows/deploy.yml@v1
    with: { environment: production }
    secrets: inherit
```

3.4 Marketplace Actions — Due Diligence

Before adopting a third-party Marketplace action: check maintenance activity (recent commits/releases), star count/adoption, whether it's GitHub-verified, and always pin to a full commit SHA rather than a mutable tag (see section 6). For anything security-sensitive (deploy credentials, signing), prefer official vendor actions over unknown community ones.

4. Runners: Hosted, Self-Hosted & Scaling

4.1 GitHub-Hosted Runners

Ephemeral VMs provisioned fresh per job, pre-installed with common tooling, torn down after the job. Pros: zero maintenance, strong isolation between jobs. Cons: limited customization, cost at scale, no access to private/on-prem networks without extra steps (self-hosted or a VPN/tunnel action).

4.2 Self-Hosted Runners

You install the runner agent on your own VM/container/bare metal. Needed for: private network/VPC access, special hardware (GPU), licensed software, or cost control at high volume. Trade-off: you own patching, scaling, and job isolation (a compromised job can potentially persist state or access other jobs on a long-lived self-hosted runner unless it's rebuilt/ephemeral per job).

4.3 Actions Runner Controller (ARC) Senior favorite

ARC runs self-hosted runners as autoscaling Kubernetes pods, spinning up ephemeral runner pods on demand per queued job and scaling to zero when idle — combining the flexibility of self-hosted runners with the ephemeral/clean-slate isolation of GitHub-hosted ones. This is the standard modern pattern for large orgs needing custom tooling or private network access without managing static VMs.

4.4 Runner Groups & Labels

Runner groups control which repos/orgs can use a set of self-hosted runners (access control); labels (`runs-on: [self-hosted, linux, gpu]`) route jobs to the correct runner pool based on required capabilities.

4.5 Runner Sizing & Cost

GitHub-hosted larger runners (more CPU/RAM) cost more per minute; choosing the smallest runner that doesn't bottleneck the job, combined with caching and parallelization, is the main cost lever. At scale, self-hosted/ARC on spot/preemptible nodes is typically far cheaper than GitHub-hosted minutes.

5. Secrets, Environments & OIDC

5.1 Secret Scopes

Scope	Description
Repository secrets	Available to all workflows in that repo
Organization secrets	Shared across repos, optionally restricted to a subset
Environment secrets	Scoped to a named environment (e.g. <code>production</code>); can require approval before a job using them runs

5.2 Environments & Protection Rules

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://app.example.com
    steps:
      - run: ./deploy.sh
```

Environments support **required reviewers** (manual approval gate), **wait timers** (mandatory delay before proceeding), and **deployment branch restrictions** (only `main` can deploy to `production`) — the primary mechanism for building approval gates natively in GHA.

5.3 OIDC Federation to Cloud Providers Senior favorite

Instead of storing long-lived AWS/Azure/GCP credentials as secrets, GitHub Actions can present a short-lived OIDC token to the cloud provider, which exchanges it for temporary credentials scoped to a specific IAM role — trust is established by a role trust policy that checks the GitHub repo/branch/environment claims in the token.

```
permissions:
  id-token: write # required to request the OIDC token
  contents: read

steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::123456789012:role/gha-deploy-role
      aws-region: us-east-1
  - run: aws s3 ls
```

Interview talking point: this eliminates an entire class of risk — leaked static cloud keys in secrets/logs — because the credentials are short-lived and scoped precisely to the workflow/branch that requested them.

5.4 Masking & Secret Hygiene

- GitHub automatically masks known secret values in logs, but this only works for exact string matches — if a secret is transformed (base64-encoded, concatenated), it won't be masked; avoid printing secrets at all, even "for debugging".
- Secrets are not passed to workflows triggered by forks on `pull_request` by default — a deliberate security boundary; don't work around it with `pull_request_target` without understanding the risk (section 6).

6. Security & Permissions Hardening

6.1 Pinning Actions to a SHA

```
# Risky: tag is mutable, can be re-pointed by the action author (compromised or malicious)
- uses: some-org/some-action@v3

# Safer: immutable reference
- uses: some-org/some-action@a1b2c3d4e5f6... # full commit SHA
```

A tag like `@v3` can be moved to point at different code at any time (accidentally or via a compromised maintainer account) — pinning to a full SHA guarantees the exact code that runs, a well-known supply-chain hardening step.

6.2 `GITHUB_TOKEN` Permission Scoping

```
permissions:
  contents: read
  pull-requests: write # only what this workflow actually needs
# everything else implicitly 'none'
```

By default the automatic `GITHUB_TOKEN` can have broad permissions; explicitly declaring a minimal `permissions:` block (principle of least privilege) limits the blast radius if a step is compromised (e.g., via a malicious dependency in `npm ci`).

6.3 `pull_request` VS `pull_request_target` Senior favorite

<code>pull_request</code>	<code>pull_request_target</code>
Runs in the context of the fork's code, with a read-only, restricted token, no access to base repo secrets	Runs in the context of the base repo with full token permissions and secrets, even though it's triggered by a fork's PR
Safe default for running tests on external PRs	Dangerous if it checks out and executes the fork's PR code directly — that is effectively running untrusted code with your secrets

Correct pattern when you must use `pull_request_target` (e.g., to comment on a PR from a fork): never execute code from the incoming PR's ref directly; if you must build/test PR code, require a maintainer to explicitly label the PR as reviewed/trusted first.

6.4 Script Injection via Untrusted Input

```
# VULNERABLE: attacker can put shell metacharacters in the PR title
- run: echo "Building for ${github.event.pull_request.title}"

# SAFE: pass untrusted input through an environment variable, never inline into the script
- run: echo "Building for $PR_TITLE"
  env:
    PR_TITLE: "${github.event.pull_request.title}"
```

Directly interpolating attacker-controlled strings (PR titles, issue bodies, branch names) into a `run:` script is a classic script-injection vector; always route untrusted values through `env:` instead of string interpolation.

6.5 Supply Chain: SBOM, Signing & Provenance

Same principles as general CI/CD supply-chain security apply here: generate SBOMs during build, sign artifacts/images with Cosign, and (for public/OSS projects) use GitHub's native **artifact attestations**/build provenance to let consumers verify an artifact was genuinely built by your workflow.

6.6 Dependabot & Automated Dependency Updates

Dependabot (native to GitHub) opens automated PRs for outdated/vulnerable dependencies (and can update GitHub Actions versions themselves via `.github/dependabot.yml` with `package-ecosystem: "github-actions"`) — a key part of keeping pinned SHAs current without manual tracking.

7. Caching, Artifacts & Performance

7.1 Cache vs Artifacts Senior favorite

actions/cache	actions/upload-artifact / download-artifact
Best-effort speed optimization (dependencies, build tool caches)	Guaranteed output of a job, needed by a later job or for download by the user
May be evicted (LRU, 10GB repo cache cap); pipeline must work correctly on a cache miss	Retained for the configured retention period regardless of eviction pressure
Keyed by a hash (e.g. of the lockfile) with restore-key fallbacks	Explicitly named and passed between jobs/workflow runs

```
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: npm-${{ runner.os }}-${{ hashFiles('package-lock.json') }}
    restore-keys: |
      npm-${{ runner.os }}-
```

7.2 Passing Data Between Jobs

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with: { name: dist, path: dist/ }

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with: { name: dist, path: dist/ }
      - run: ./deploy.sh
```

7.3 Reducing Pipeline Duration

- Parallelize independent jobs; split large test suites across matrix shards.
- Cache dependencies and Docker layers (`docker/build-push-action` supports GitHub Actions cache backend for BuildKit layers).
- Use `concurrency:` groups to cancel superseded runs on the same branch/PR rather than letting stale runs finish.
- Use path filters so unrelated changes in a monorepo don't trigger unrelated workflows.
- Prefer JavaScript/composite actions over Docker actions where startup latency matters.

7.4 Concurrency Control

```
concurrency:
  group: deploy-${{ github.ref }}
  cancel-in-progress: true
```

Prevents two deployments to the same environment/ref racing each other, and cancels an outdated in-flight run when a newer commit supersedes it — important both for safety (no overlapping deploys) and for cost (don't pay for compute on a run you no longer care about).

8. Integrations

8.1 Docker & Container Registries

```
- uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}

- uses: docker/build-push-action@v6
  with:
    push: true
    tags: ghcr.io/${{ github.repository }}:${{ github.sha }}
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

8.2 Kubernetes & GitOps

Two common patterns: (1) direct deploy — a workflow authenticates to the cluster (via OIDC-federated cloud credentials + `aws eks update-kubeconfig` or similar) and runs `kubectl apply` / `helm upgrade`; (2) GitOps — the workflow only updates an image tag in a separate manifest/Helm-values repo, and ArgoCD/Flux (already running in-cluster) detects and reconciles the change. Pattern (2) is preferred at scale since it avoids handing cluster credentials to CI at all.

8.3 Terraform

```
- uses: hashicorp/setup-terraform@v3
- run: terraform fmt -check
- run: terraform init
- run: terraform plan -out=tfplan
- name: Post plan to PR
  uses: actions/github-script@v7
  with:
    script: |
      github.rest.issues.createComment({ ... })
- run: terraform apply tfplan
  if: github.ref == 'refs/heads/main'
```

8.4 Cloud Providers

All three major clouds have official OIDC-based auth actions: `aws-actions/configure-aws-credentials`, `azure/login`, `google-github-actions/auth` — each preferred over storing static keys.

8.5 Notifications / ChatOps

Slack/Teams notifications on failure (via webhook actions), auto-commenting on PRs with test coverage/plan output (`actions/github-script`), and auto-labeling/triage bots are common uses of GHA beyond pure build/deploy.

8.6 Code Quality & Security Tool Integrations

SonarQube/SonarCloud, CodeQL (native GitHub SAST), Snyk, Trivy, and OWASP ZAP all have official actions and typically publish results to the **Security tab** (via SARIF upload) so findings appear directly in the GitHub UI alongside code.

9. Monorepos, Concurrency & Cost Optimization

9.1 Monorepo Patterns

- **Path filters** (`on.push.paths`) so each service's workflow only runs when its own directory changes.
- **Reusable workflows** per "workflow shape" (e.g., a generic `node-service-ci.yml`) called by many thin per-service caller workflows — avoids duplicating pipeline logic 50 times.
- Change-detection actions (e.g., `dorny/paths-filter`) to compute which services changed and dynamically build a matrix of affected services in a single workflow run.

9.2 Cost Optimization Levers

- Right-size runners (don't default to the largest runner for every job).
- Aggressive caching of dependencies and Docker layers.
- Cancel superseded runs via `concurrency: .`
- Move self-hosted/ARC runners to spot/preemptible capacity for non-time-critical jobs.
- Split "fast feedback" jobs (lint, unit test) from "slow" jobs (E2E, load test) so most PR iterations only pay for the fast path.

9.3 Org-Wide Governance

At scale, platform teams enforce standards via: required reusable workflows (org policy that certain checks must run), required status checks in branch protection, org-level Actions permissions (restrict which actions/Marketplace entries can be used org-wide, allow-list only), and centralized self-hosted runner fleets (ARC) shared across teams with runner groups for access control.

10. Common Errors & Troubleshooting Playbook

Error: `Error: Process completed with exit code 1` with no other detail.

Fix: Re-run with debug logging enabled (set repo secrets `ACTIONS_STEP_DEBUG=true` and `ACTIONS_RUNNER_DEBUG=true`) to see verbose step-by-step output; check the actual failing command's own output above the generic exit-code line rather than treating this message as the root cause.

Error: Workflow doesn't trigger at all on push.

Fix: Check `on.push.branches` / `paths` filters aren't excluding this commit, confirm the workflow file is on the default branch (workflow definitions on feature branches only apply to events on that same branch context in most cases), and check Actions isn't disabled for the repo/org.

Error: Secret shows as empty string in a forked-repo PR workflow.

Fix: Expected behavior — secrets are not passed to `pull_request` workflows triggered by forks, by design, to prevent fork PRs from exfiltrating secrets. Use `pull_request_target` only with extreme care (section 6.3), or split the workflow so the sensitive step only runs after merge/on a trusted trigger.

Error: `remote: Permission to org/repo.git denied / GITHUB_TOKEN` insufficient permissions when pushing or calling the API.

Fix: The auto-generated token's `permissions:` block is too restrictive for the action being attempted (e.g., pushing a tag needs `contents: write`); add the specific scope needed, or use a fine-grained PAT stored as a secret for cross-repo operations the default token can't do.

Error: Cache never hits ("Cache not found for input keys").

Fix: The cache key (often a hash of a lockfile) changed, the cache exceeded the 10GB per-repo limit and got evicted (LRU), or the cache was scoped to a different branch than expected (cache is only shared/fallback-restored from the same branch or its base branch by default). Verify the `hashFiles()` input path is correct and check the repo's Actions cache usage page.

Error: Job stuck "Waiting for a runner to pick up this job" indefinitely (self-hosted).

Fix: No self-hosted runner online with the matching labels, ARC autoscaler hasn't provisioned a pod (check controller logs/HPA metrics), or the runner registered to the wrong repo/org scope. Verify labels in `runs-on:` exactly match a registered/available runner.

Error: Docker build step fails with `failed to solve: process "/bin/sh -c ..." did not complete successfully`.

Fix: Standard Docker build failure — read the actual layer/command output above; common root causes are a missing build-arg/secret, network-restricted base-image pull inside a corporate self-hosted runner, or an architecture mismatch (building an ARM-only step on an x86 runner without `--platform`).

Error: Matrix job succeeds individually but the "required status check" in branch protection never turns green.

Fix: Branch protection requires an exact job name; if the matrix job name is templated (e.g., includes `${{ matrix.node }}`), each combination reports a differently-named check, and the protection rule may be pointing at a job name that no longer exists after a matrix change. Use a final "all matrix jobs passed" aggregator job as the actual required check.

Error: Reusable workflow fails with `secrets not found` even though the caller has the secret.

Fix: Secrets aren't automatically passed into called reusable workflows; you must explicitly pass them (`secrets: inherit` or an explicit `secrets:` map in the caller) and declare them in the called workflow's `on.workflow_call.secrets`.

10.1 General Debugging Method

1. Open the raw log of the failing step, not just the summary annotation.
2. Enable step/runner debug logging for a re-run if the failure isn't self-explanatory.
3. Check what changed: workflow YAML diff, action version bump, dependency lockfile, runner image update.
4. Reproduce locally with `act` (a local GitHub Actions runner emulator) where feasible for fast iteration.
5. Check scope/permission boundaries first for anything involving secrets/tokens/forks — most "mysterious" GHA failures are permission-boundary related.

11. Tricky Questions & Traps

Q: Do jobs in the same workflow file run in the order they're written?

No — jobs run in parallel by default. Only `needs:` creates ordering. Steps within one job are sequential; jobs are not, unless explicitly chained.

Q: If you set a repository secret, is it safe automatically for any workflow that runs?

Not for fork-triggered `pull_request` workflows (secrets withheld by design), but it IS available to `pull_request_target` workflows even when triggered by a fork — which is exactly why `pull_request_target` combined with checking out and running fork code is a critical vulnerability pattern.

Q: Is caching a dependency the same as guaranteeing it will be there next run?

No — cache is best-effort with LRU eviction and a per-repo size cap; a pipeline that "requires" the cache to function correctly is fragile. Correct design: cache misses should only cost time (re-download/rebuild), never break the pipeline.

Q: You pinned an action to `@v4` — is that sufficiently secure against supply-chain risk?

Trap — a version tag like `v4` is typically a moving major-version tag that can be repointed by the action's maintainer (or an attacker who compromises their account) to different code without you noticing. Only a full commit SHA is immutable.

Q: A workflow needs to deploy to AWS — is storing an AWS access key as a GitHub secret best practice in 2026?

No, not as a first choice — OIDC federation (short-lived, per-run credentials via a cloud IAM trust policy) is the recommended modern approach; static keys are a fallback for providers/cases where OIDC isn't available.

Q: If a step fails partway through a job, do later steps in that job still run?

By default, no — the job stops and subsequent steps are skipped, unless a later step explicitly has `if: always()` or `if: failure()` / `if: cancelled()`.

Q: Does `actions/checkout` fetch the full git history by default?

No — it does a shallow clone (depth 1) by default for speed; if your workflow needs full history (e.g., to diff against a previous tag, or for tools like semantic-release), you must set `fetch-depth: 0` explicitly.

Q: Self-hosted runners are "free" compute since you already own the hardware — any hidden costs?

Trap — you now own patching, scaling, isolation between jobs (risk of cross-job contamination on long-lived runners), and availability/on-call for the runner fleet itself; "free" compute has real operational cost that must be weighed against GitHub-hosted minutes pricing.

Q: Is a passing required status check proof the code is safe to merge?

Only as strong as the checks configured — a mis-scoped branch protection rule (e.g., pointing at a stale matrix job name, per section 10) can make "required checks" silently not actually gate anything meaningful; always verify the required check list still matches real job names after refactors.

12. Expected Interview Q&A (Basic → Advanced)

12.1 Basic Level

Q: What is a workflow, a job, and a step?

A workflow is a YAML automation file triggered by events; a job is a group of steps run together on one runner; a step is a single command or action invocation within a job.

Q: What's the difference between `run:` and `uses:` in a step?

`run:` executes a shell command directly; `uses:` invokes a reusable action (JavaScript, Docker, or composite).

Q: What triggers can start a workflow?

Repository events (push, pull_request, release, issue events), manual dispatch (workflow_dispatch), scheduled cron (schedule), external API calls (repository_dispatch), and calls from other workflows (workflow_call).

Q: How do you pass data from one step to a later step?

Write to `$GITHUB_OUTPUT` in the earlier step and reference it via `steps.<id>.outputs.<name>` in a later step of the same job.

Q: What is a matrix build?

A strategy that runs the same job multiple times with different variable combinations (OS, language version, etc.) in parallel, useful for cross-platform/cross-version testing.

12.2 Intermediate Level

Q: How do you share pipeline logic across many repositories?

Reusable workflows (`workflow_call`) for sharing entire jobs/pipeline shapes, or composite actions for sharing a sequence of steps within one job; both live in a central repo and are versioned/referenced by tag or SHA.

Q: How would you set up a manual production deployment with approval?

Define a `production` environment with required reviewers configured in repo settings, then have the deploy job target `environment: production` — the job pauses for approval before running.

Q: How do you avoid running duplicate/overlapping workflow runs on rapid pushes to the same PR?

`concurrency:` groups keyed on the ref/PR number with `cancel-in-progress: true`, so only the latest push's run proceeds and older superseded runs are cancelled.

Q: What's the safest way to authenticate a workflow to AWS/Azure/GCP?

OIDC federation with a scoped IAM/identity role trust policy, using `id-token: write` permission and the provider's official auth action, instead of static long-lived access keys.

Q: How do you handle secrets needed by a workflow triggered from a forked repository's PR?

By design they aren't available on `pull_request`; the safe pattern is to run untrusted/build steps without secrets on `pull_request`, and only run secret-requiring steps (e.g., posting results, deploying preview environments) in a separate, trusted trigger (post-merge, or a second workflow triggered by `workflow_run` after the first completes) that never executes the fork's code directly.

12.3 Advanced / Senior Level

Q: Design a CI/CD setup for an organization with 100+ repositories that all need consistent security scanning and deployment gates.

Central "shared workflows" repo with org-approved reusable workflows for build/test/scan/deploy; org-level required workflow policy so individual repos can't opt out of baseline security checks; org Actions permissions allow-listing which third-party actions can be used; ARC-based centralized self-hosted runner fleet with runner groups for team-level isolation; and a platform team owning the shared workflow templates as their own versioned, tested product.

Q: How would you migrate from Jenkins to GitHub Actions for a large legacy pipeline without a big-bang cutover?

Run both in parallel temporarily (Jenkins triggers a GHA workflow via `repository_dispatch` /API, or vice versa, during transition), migrate one team/service at a time starting with lowest-risk pipelines, extract shared Groovy shared-library logic into GHA reusable workflows/composite actions, and keep Jenkins as source of truth only until the equivalent GHA pipeline has proven itself over several real releases before decommissioning.

Q: How do you prevent a compromised third-party action from exfiltrating your deployment secrets?

Pin every action to a full commit SHA (not a tag), scope `GITHUB_TOKEN` /OIDC role permissions minimally per workflow so even a compromised step has limited blast radius, avoid passing secrets to steps/actions that don't need them, use org allow-listing to restrict which actions can be used at all, and consider network egress restrictions on self-hosted/ARC runners to limit where a compromised step could send data.

Q: Your GHA-based deployment pipeline needs to support canary releases with automatic rollback based on live metrics. How would you architect this?

GHA itself isn't a progressive-delivery engine; the correct architecture is GHA builds/tests/scans and updates a GitOps manifest repo with the new image tag behind a canary weight, then a progressive-delivery controller in-cluster (Argo Rollouts/Flagger) does the actual traffic shifting and automated metric-based promotion/rollback, decoupled from the CI tool entirely — a common trap is trying to hand-roll canary logic in workflow YAML instead of using purpose-built in-cluster tooling.

13. System Design Style Questions

Structure: clarify requirements → propose workflow/repo architecture → call out trade-offs → mention security/cost/observability.

Q: Design the CI/CD setup for a monorepo with 30 microservices using GitHub Actions.

Talk through: path-filtered triggers or a change-detection job that computes affected services, a reusable workflow template per service "type" (e.g., node-service, go-service) called by thin per-service workflows, a shared runner pool (ARC) sized for peak concurrent PR activity, and a single GitOps manifest repo per environment that all service pipelines update — keeping the deployment mechanism uniform even though build steps differ per service.

Q: Design a secure pipeline for building and deploying a container that will run with elevated privileges in a regulated (e.g., financial/healthcare) environment.

Talk through: SAST + SCA + container scan as blocking gates, SBOM generation, image signing (Cosign) with cluster-side admission enforcement (only run signed images), OIDC-only cloud auth (no static keys), mandatory required-reviewer approval on the production environment, immutable action pinning by SHA, and full audit logging of who approved what deployment for compliance evidence.

Q: Your org's GitHub Actions bill has grown 5x in six months with no corresponding growth in deploys. How do you investigate and fix it?

Talk through: pull Actions usage reports to find the top-consuming workflows/repos, look for missing **concurrency:** cancellation causing redundant runs, oversized runners used for trivial jobs, cache misses forcing full rebuilds repeatedly, and matrix explosions testing unnecessary combinations; fix via right-sizing runners, adding path filters, fixing cache keys, and moving heavy/infrequent jobs (E2E, load tests) off the PR-blocking critical path.

14. Behavioral / Experience-Based Questions

Prepare STAR-format stories from your own project history for each:

- "Describe a GitHub Actions pipeline you built or improved. What was the before/after impact on build time, deploy frequency, or failure rate?"
- "Tell me about a security issue you found or prevented in a CI/CD pipeline (e.g., a secret leak, an unpinned action, an overly broad token)."
- "Describe a time a workflow's caching or concurrency behavior caused an unexpected production issue, and how you diagnosed it."
- "How have you standardized CI/CD practices across multiple teams/repositories?"
- "Tell me about migrating a pipeline from one CI tool to GitHub Actions (or vice versa) — what was hardest?"

15. Quick Reference Cheat Sheet

15.1 Common Snippets

```
# Manual trigger with inputs
on:
  workflow_dispatch:
    inputs:
      environment:
        type: choice
        options: [dev, staging, prod]

# Conditional job based on branch
if: github.ref == 'refs/heads/main'

# Run regardless of prior failure
if: always()

# Full git history checkout
- uses: actions/checkout@v4
  with: { fetch-depth: 0 }

# OIDC AWS auth
permissions: { id-token: write, contents: read }
- uses: aws-actions/configure-aws-credentials@v4
  with: { role-to-assume: arn:aws:iam::ACCOUNT:role/ROLE, aws-region: us-east-1 }

# Concurrency cancellation
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

15.2 One-Line Definitions to Have Ready

Workflow	YAML automation file triggered by repo/external events
Reusable workflow	Shares whole jobs across repos via <code>workflow_call</code>
Composite action	Shares a sequence of steps within one job
ARC	Actions Runner Controller — autoscaling self-hosted runners on Kubernetes
OIDC federation	Short-lived cloud credentials issued per run, no static keys
<code>pull_request_target</code>	Runs with base-repo secrets/permissions even for fork PRs — high risk if misused
Concurrency group	Cancels/prevents overlapping runs for the same key (branch/ref)
SHA pinning	Referencing an action by immutable commit hash instead of a mutable tag

15.3 Day-Before-Interview Checklist

- Re-read section 6 (Security) and section 5.3 (OIDC) — these differentiate senior candidates.
- Be ready to explain `pull_request` vs `pull_request_target` from memory without notes.
- Rehearse 2-3 STAR stories (section 14).
- Review the tricky Q&A (section 11).
- Know exactly how your own recent projects handle secrets, runners, and deployment gating.